

Simulation-Driven Development: Building Hardware with AI Agents Against a One-to-One, No-Mocks Simulation

Nils Achermann

Abstract

Letting AI agents write large parts of an embedded or hardware system removes the property that once made such work tractable: a single engineer holding the complete model of the system in mind. This paper describes *simulation-driven development*, a methodology that relocates that model out of the engineer’s head and into a software simulation that reproduces the target hardware’s interfaces and behaviour one-to-one, with no mocks, then lets a gated multi-layer agent pipeline build, test, and repair against it. The method rests on four ideas. First, every handoff between models is a lossy compression boundary—as in human communication, where one conveys only a part of what one knows and the rest is held as sensation or optimized away below awareness, yet remains reachable—so the pipeline is built to minimize and contain that loss. Second, the simulation uses no mocks: rather than approximate a peripheral, the real compiled driver is loaded and exercised through true one-to-one transactions. Third, progress is gated by a review-key system in which a producing agent can never grant its own advance; only a reviewer may. Fourth, the work proceeds in two stages: a software simulation first, then on-device simulation in which the pipeline controls power, reset, inputs, and even observes the screen through a framebuffer dump and a camera, with a hardware watchdog that can recover an unrecoverable device. The result is a development loop in which the more it iterates, the more defects are caught before they reach silicon. The methodology is presented with its architecture, its agent roles, and the discipline that keeps a long automated run from drifting away from the engineer’s intent.

1 Introduction

Simulation-driven development means giving AI agents, and the automated systems around them, as much control over a hardware system and its code as possible, so that they can control and debug every aspect of it. The idea is not new in spirit. Field-programmable gate array (FPGA) work has long been simulation-driven, and for a concrete reason: a designer hands a pipeline of logic to the tools and, to a large degree, hopes for the best. One does not

really know what will result, where every signal is routed, whether every path works, whether each signal arrives at the right time and is properly formed. The design becomes complex very quickly, and past a certain size it is genuinely unclear whether all of it works together. Proceeding by careful timing and hope is not efficient, so the design is simulated first, because simulation is the only place the behaviour can be seen before commitment to silicon.

Ordinary programming used to be different. In MicroPython, C, C++, or Python, the engineer knew exactly what had been written and held a complete overview of the system. That property quietly disappears once AI writes large parts of the code, and it disappears faster as the work scales: multi-layer agentic systems and multi-layer pipelines amplify the effect, because several models now write and transform code across many handoffs, and no single human ever read all of it.

The method therefore starts from an honest division of labour. The engineer designs the key aspects of the system—the cores, the parts that determine what the whole thing is—and then hands the work off, in parts, to agents. The simulation restores the overview that was lost: instead of holding the entire system in mind, the engineer holds the architecture, and the simulation faithfully holds the detail, so that any part of it can be inspected at any time. The remainder of this paper makes that arrangement precise. Section 2 states the motivation and the central assumption that every model handoff compresses information. Section 3 describes the multi-layer agent pipeline that does the building. Sections 4 and 5 describe the two simulation systems: software first, then on-device. Section 6 discusses the principles that keep the method anchored, and Section 7 concludes.

2 Background and Motivation

The lighter alternative to this method—a single self-managing agent pipeline working against a written ruleset—is adequate when a system has one reasonably simple architecture and can be built more or less in a single pass. Simulation-driven development earns its additional cost at the point where two conditions hold at once: the system must be developed feature by feature,

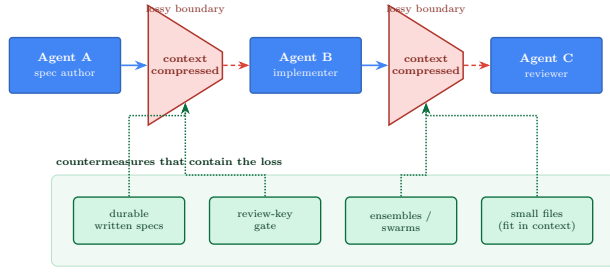


Figure 1: Each model-to-model handoff compresses the shared context (the funnels). Four countermeasures—durable written specifications, the review-key gate, redundant ensembles, and files small enough to fit whole in context—contain the loss.

and it can no longer be built all in one pass. That is the regime in which drift, lost context, and unseen hardware behaviour compound.

2.1 Handoffs as lossy compression

The design decisions that follow rest on one observation about how models pass work to each other: every connection between two models compresses information. The analogy is human-to-human communication. A person tells another only part of what they know; some knowledge is stored as sensation and some is optimized away below awareness, yet remains reachable without deliberate effort. A model handoff behaves the same way: only part of the context survives the boundary, and the receiver then acts confidently on the compressed version. This is the author’s working model rather than a single published result, but the surrounding evidence is concrete: long contexts are used unevenly, with information in the middle attended to least [1], and multi-agent systems fail in characteristic ways, including miscommunication between stages and errors that cascade as one stage’s small mistake becomes the next stage’s confident input [2]. Every structural choice in the method—the durable written specifications, the review-key gate, the redundant ensembles, and the files kept small enough to fit whole in context—exists to make this loss small, visible, and caught before it compounds (Figure 1).

3 The Multi-Layer Agent Pipeline

The pipeline is the substrate that performs the building; simulation-driven development points it at a faithful simulation and wraps it in a loop. It draws on two kinds of model. Hosted frontier models from the major providers perform the heavy reasoning and the bulk of implementation. A local fleet of open-weight models—for example quantized members of the Qwen family for general and code work [18], and Gemma 3 [19]—carries the high-

volume, lower-stakes load of monitoring long automated runs and indexing their output for later recall. A local fleet is made practical by layer-streaming inference, which runs a large model on a small accelerator by loading and offloading one layer at a time [16], and by quantization of weights and the attention cache [17].

3.1 The agent hierarchy and its dedicated teams

The pipeline is not a flat pool of interchangeable agents but a managed hierarchy in which every node has a narrow remit (Figure 2). At the top sits the engineer, who owns the architecture and is the final authority on intent (Section 6). Beneath the engineer an orchestrator turns the architecture into a concrete plan—a task list, the module decomposition, and the key architectural decisions, all written down—then delegates each part to a specialized team and reconciles what the teams return. This is the orchestrator–worker pattern [9]; the discipline that makes it work here is *dedication*. Every agent has one job and only that job, because an agent with a single narrow responsibility has far less room to drift than a generalist asked to design, instrument, and judge at once. The same separation that forbids a producer from reviewing its own work (Section 3.3) applies team by team across the whole hierarchy.

Five kinds of team hang off the orchestrator. The *implementation team* does the building and is itself split by responsibility into timing-and-architecture agents, processing agents, and debug agents, whose non-overlapping jobs are detailed in Section 4.2; the management point is only that the agent shaping the structure is never the agent that instruments it or the agent that judges it. The *simulation’s own builder team* is kept separate from the agents that build product features, because the simulator is itself software that must be extended, and most of what it needs is discovered only by running it; this team owns extraction, simulation enhancement, and true-to-hardware review, so the simulator can grow without drifting from the hardware it mirrors (Section 4). A long-lived *management engine* carries one duty—keeping the simulation current with every bug found later, including those found only on the device (Section 5)—because currency is what keeps the human-owned architecture trustworthy (Section 6). The *review chain* of at least three independent AI reviewers and a final human gates every advance through the review key (Section 3.3). Finally, the *local fleet* runs as swarms that monitor the long runs, index their output for later recall, and form the two-module drop filter described next.

3.2 Swarms and the drop filter

The local fleet is run in ensembles rather than as single models answering once. Sampling several independent attempts and reconciling them cancels much of the random,

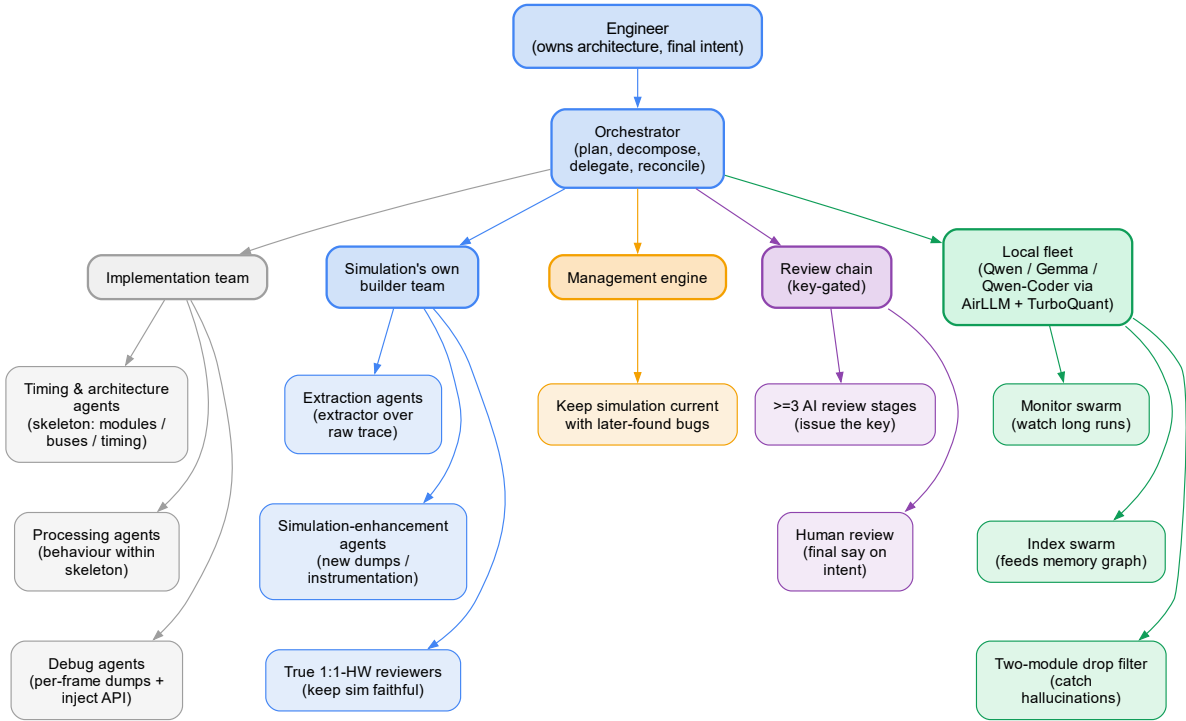


Figure 2: The agent hierarchy as a management tree: the human-owned architecture at the root, an orchestrator that plans and delegates, and the dedicated teams beneath it—the implementation team (timing-and-architecture, processing, and debug agents), the simulation’s own builder team, the management engine, the gated review chain, and the local fleet—each leaf naming a concrete duty. Every agent has a single, narrow responsibility.

variance-driven error a single handoff would carry forward; this is the self-consistency effect [3]. Outputs then pass a two-module architecture in which one module produces and a second checks and filters, so most hallucinations and low-quality outputs are dropped at that boundary before reaching the human. Ensembling reduces variance error; it does not remove correlated or systematic mistakes, which is precisely why explicit review stages and a human follow.

3.3 The review-key gate

Per-project rules files are necessary but not sufficient: a model told not to skip a review can still decide mid-run that it has done enough. The pipeline removes that freedom with a key system. No model may advance past a review, or skip it, on its own authority; advancing requires a key, and a key may be issued only by a reviewer model, never by the model whose work is under review (Figure 3). Progress is not a claim the worker makes but a permission the reviewer grants. This makes non-bypassable the now-standard practice of using a separate model to judge another model’s output [4]. Implementa-

tion passes through at least three independent AI review stages, each gated by a key, and then a final human stage; stacking independent reviewers catches the compounding errors that multi-stage model systems are prone to [2, 5], and iterative self-refinement between stages improves the output further [6, 7]. The human stage is the only one that reliably catches drift away from intent rather than mere defects in execution, a long-standing requirement of trustworthy interactive systems [8]. The shape is the orchestrator-worker pattern, in which a lead process delegates to parallel specialized workers and a later pass reconciles their results [9].

4 Software Simulation Architecture

Before any feature is built, two systems are set up, of which only the first is used at the start: a software simulation side, and an on-device side (Section 5). Figure 5 shows the software simulator at a glance.

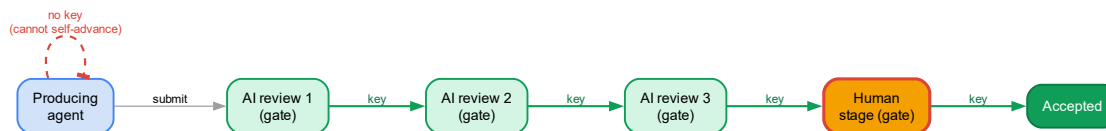


Figure 3: The review-key gate. A producing agent cannot advance on its own claim of completion; only a reviewer issues the key that unlocks the next stage. Work passes at least three independent AI review stages and a final human stage; the producer’s attempt to grant its own advance (red, dashed) is forbidden.

4.1 From datasheets to one-to-one specifications

The software side begins with strictly human-gated knowledge gathering: every datasheet and erratum is read, written down, and verified by hand, then verified again by the AI review stages, because a faithfully wrong simulation lies confidently on every loop. Each hardware aspect is then specified one-to-one, first as a Markdown document that is the contract, and the aspects are split by performance budget: fast simulations are written in Julia or C, slower ones in Python.

4.2 Three agent roles

The software simulation is built by three kinds of agent with narrow, non-overlapping jobs, the implementation team of Section 3.1. *Timing and architecture agents* build only the structural skeleton (modules, buses, and timing) so that the shape and timing match the hardware, and nothing else; they deliberately implement no behaviour. *Processing agents* then implement the behaviour within that fixed skeleton, never altering the structure they were handed. *Debug agents* are the most important role, because the entire method depends on the simulation being observable and controllable to the finest grain: a loop that cannot see what the simulation did, or drive what it does next, is blind, and no-mocks fidelity is wasted if the result cannot be inspected. In the Python paths these agents apply dumps at the finest granularity—every timing frame and every pipeline stage—and expose an interface through which other agents can inject inputs (for example button presses) and select exactly what is recorded. In the Julia or C paths, where per-call instrumentation would be too costly, a debug agent instead declares in a header file a large block of writable array slots that the running code consumes (so that later agents can drive interface inputs by writing those slots), and every step is written to a single complete dump file.

4.3 Extraction, and keeping the raw trace

The same agent that built the simulation, retaining full context, then writes a Python extraction program that turns the large dump into structured, queryable form. The raw dump is kept regardless, because later debugging routinely needs correlations no one anticipated, and the only way to obtain such signal-level correlations is for an agent to write a new extractor against the original trace. Extractors are disposable views; the raw trace is ground truth.

4.4 The no-mocks law

The central rule is that the simulation stays one-to-one with the hardware, with no mocks, ever. A serial bus illustrates it: writing a quick mock of a peripheral in a scripting language is far easier, but a mock only proves that code works against an *assumption* of the hardware. To find the defects that bite on the device, the bus is written for real in C, compiled, loaded as a shared library, and driven through true one-to-one transactions [11, 12]. This is the hardware analogue of the established testing principle that high-fidelity doubles catch the bugs convenient mocks hide [10], applied without exception even when the faithful path is a large detour. The detours are where defects that would otherwise reach silicon are caught instead. The one-to-one claim is scoped to the interfaces and behaviour reproduced in software by exercising the real driver; a host process is not the silicon, which is exactly why the on-device stage (Section 5) follows, to close the remaining gap in timing and analog behaviour.

4.5 The loop, and the simulation’s own builders

With the engine standing and documented, two long-lived roles run for the rest of the project: a management engine whose sole purpose is to keep the simulation current with every later-found bug, and the programming part that builds features against it. The build proceeds feature by feature under a per-file code-length limit so that any file

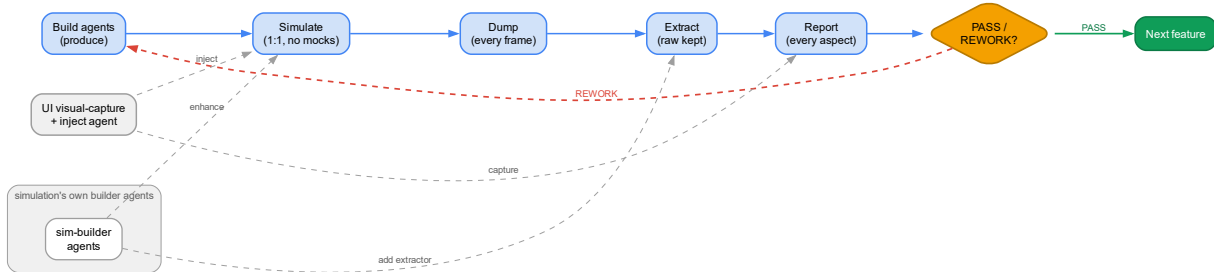


Figure 4: The software simulation loop as a parallel state diagram. Build agents produce; the simulation runs one-to-one with no mocks; every frame is dumped and extracted; a report drives a PASS or REWORK decision, with rework routed back outside the main row. A parallel lane exercises the user interface by visual capture and input injection, and the simulation maintains its own builder agents.

fits whole in a model’s context rather than being handed on as a lossy summary of itself. Each feature cycles through: build agents self-review, the simulation runs the no-mocks model and produces a comprehensive report, and the build agents decide PASS or rework (Figure 4). This self-review is the inner build-loop iteration; advancement across the independent review stages remains gated by the review key (Section 3.3), which no producing agent can issue for itself. Code is not the only thing simulated: a user interface is exercised by a visual-capture and input-injection agent that captures what is actually rendered, because rendering faults such as screen artifacts are hard to detect from values alone, even with complete buffer dumps, a reflection of the visual-regression testing principle of inspecting the rendered output, not only the buffer that produced it [13]. Because the simulation is itself software that must be built and extended—many of its needs are discovered only by running it—it has its own dedicated builder agents for extraction, simulation enhancement, and true-to-hardware review, so that every enhancement stays as near the hardware as the original engine. The loop runs until every task is worked down; the more it iterates, the more defects are caught before the on-device stage.

5 On-Device Simulation

When the software simulation stops finding new problems and the design is believed correct, the on-device system comes online. All hardware is wired so the pipeline can drive every controllable aspect—inputs, power, reset, and boot mode—because anything a human would otherwise reach over and do by hand becomes a point at which the automated loop stalls (Figure 6).

5.1 Control surface and the pin-file convention

A separate control microcontroller owns the pins that press buttons, switch power, and set modes. Its pins are organized as one file per pin plus a single index file that documents them all, so that an agent picking up the hardware knows unambiguously what each pin does and how to drive it. Buttons are driven by a single pin each; the pin file records whether the button is wired directly or through a relay, the relay case closing the real switch under program control when more hardware sits behind it. An Arduino is a fitting control microcontroller—it need not be fast, its behaviour is well documented, and on most boards opening the serial port auto-resets it into the bootloader so no manual button press is needed to flash—though any microcontroller works provided it is documented for the handoff.

5.2 A capacitor-discharge watchdog

A looped pipeline controlling real hardware will eventually drive the device into a state it cannot escape. The method plans for this with a deliberately crude, robust watchdog that does not depend on the device or the pipeline being healthy. To recover, the pipeline first charges a capacitor through a resistor—an arming step that raises it to a known voltage V_0 , read back by an analog-to-digital converter (precision is irrelevant)—and then powers the device down. With the device off, the capacitor discharges through a bleed resistor, and that discharge is the timer: its voltage follows the ordinary RC decay $V(t) = V_0 e^{-t/RC}$, with the time constant RC setting the interval. An independent detector—with its own logic, on its own power drawn from the in-between rails ahead of this stage—powers the device back on once the falling voltage drops below a set threshold. The device thus power-cycles on the capacitor’s own schedule,

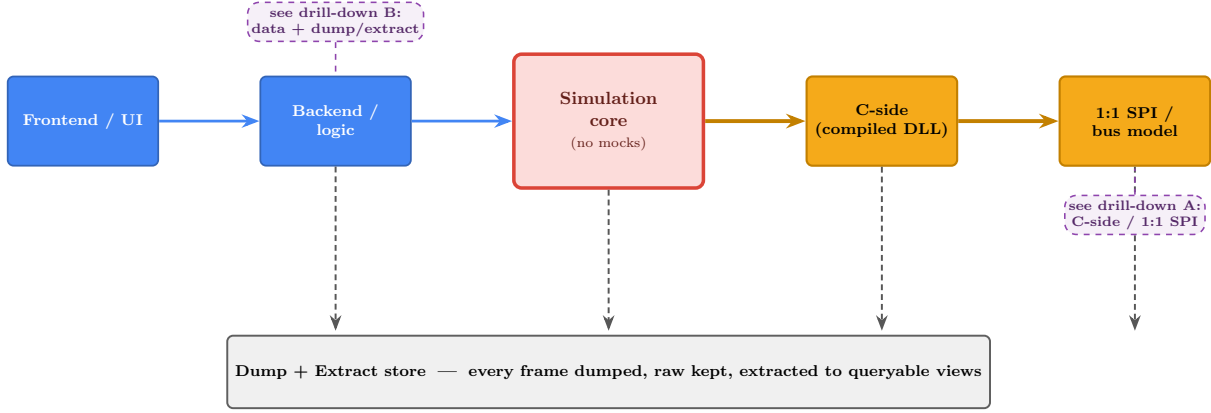


Figure 5: Top-level module map of the simulator: frontend, backend, the simulation core, the compiled C side loaded as a shared library, the true one-to-one bus path, and the dump-and-extract store. Detail is deferred to the companion manual’s drill-down figures rather than crammed here.

with no working firmware and no human hand required. A watchdog that shares the fate of what it watches is no watchdog; isolating its power and logic is what makes it trustworthy.

5.3 Observing the display

Where the device has a display, two complementary channels let the pipeline see it. A framebuffer dump in firmware serializes the pixel memory over a link so the host can reconstruct the intended image. A small camera pointed at the physical panel captures what is truly shown, including faults that occur after the framebuffer: panel glitches, refresh artifacts, a controller that does not draw what the framebuffer says. Pairing the intended image with the actual image makes their difference a defect signal in itself, compared for example by structural similarity, with optical character recognition for on-screen text. The same looped, gated pipeline then runs against the device, and every bug found is folded back into the software simulation so the two stay in agreement; Figure 7 summarizes which role owns what on the device side.

6 Discussion

The architecture sits on top. A long automated run drifts: each step is locally reasonable, each handoff loses a little context, and after enough steps the system has forgotten its link to the whole. The defence is to keep the architecture human-owned and above everything the agents produce. Generated code serves the architecture; the architecture is not rewritten to accommodate generated code. This is why currency matters: once the written architecture, the specifications, or the simulation fall out of date with the real system, the top layer stops being trustworthy and drift has nothing to push against.

Keeping them current is not documentation hygiene but the anchor of the method.

Memory shaped as a graph. The facts that matter across a long project are related—a register affecting a behaviour, a timing domain crossing a bus. A flat store forces a later agent to re-derive those relations, and large contexts are read unevenly [1]. Storing entities and their relations in a knowledge graph turns recall into a traversal; graph-structured retrieval improves reasoning over large interconnected bodies of information [14], and memory-augmented agents extend effective memory beyond the context window [15]. The graph is where the unanticipated correlations live, so that finding them later is a query rather than an excavation.

Honesty about limits. Ensembling and review reduce error but do not eliminate it; graph memory improves recall but does not guarantee correctness; and no degree of automation removes the need for the human stage. The method is a discipline for catching most problems early and cheaply, not a proof that none remain. This paper presents the methodology and its architecture; a quantitative evaluation—defect-escape rates, iteration counts, and cost—is deliberately left to future work.

7 Conclusion

Simulation-driven development addresses a specific cost of building hardware with AI agents: the loss of the single complete mental model that once made the work tractable. It relocates that model into a one-to-one, no-mocks simulation, lets a gated multi-layer pipeline build and test against it, and then extends the same pipeline to the real device with total control, direct observation of the screen, and a hardware watchdog for recovery. Two

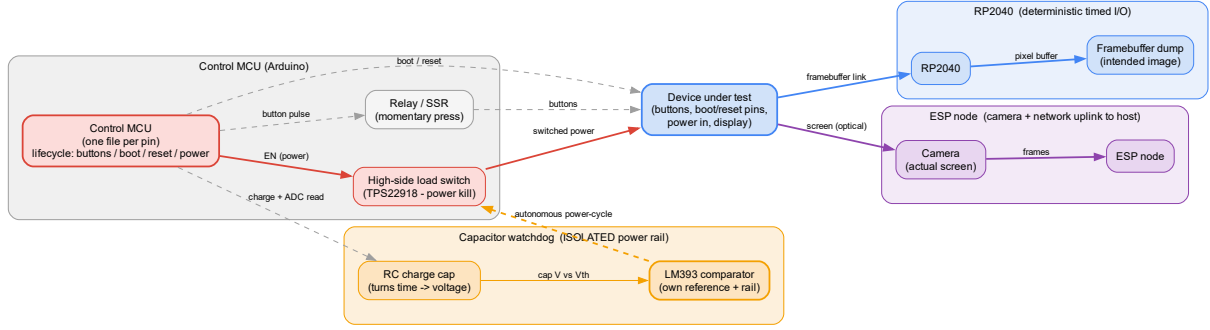


Figure 6: The on-device hardware map. A control microcontroller drives the device under test (buttons through relays, boot, reset, and a high-side power kill); one microcontroller serializes the display framebuffer (the intended image) and another captures the panel with a camera (the actual image); a capacitor-discharge watchdog on an isolated rail, with an independent comparator, power-cycles a wedged device. Data, control, and power/recovery paths are colour-coded.

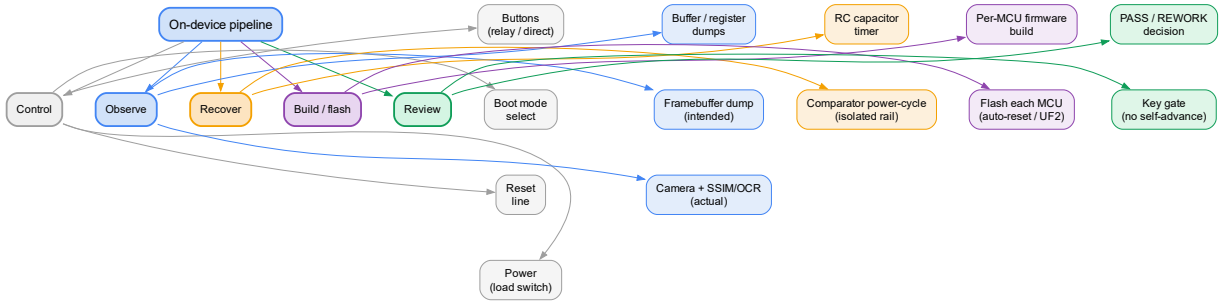


Figure 7: Responsibility tree for the on-device side: what is controlled, observed, recovered, built and flashed, and reviewed, and which role owns each.

disciplines carry most of the weight: the no-mocks law, which keeps the simulation honest, and the review-key gate with a human at the end, which keeps a long automated run from drifting. The payoff is a loop whose value grows with iteration, moving defects out of expensive on-device debugging and into cheap, observable simulation. The methodology is independent of any single tool; the companion manual details the concrete tooling and operational procedure for each stage.

References

- [1] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *arXiv:2307.03172*, 2023.
- [2] M. Cemri et al., “Why Do Multi-Agent LLM Systems Fail?” *arXiv:2503.13657*, 2025.
- [3] X. Wang et al., “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *arXiv:2203.11171*, 2022.
- [4] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *arXiv:2306.05685*, 2023.
- [5] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” *arXiv:2308.00352*, 2023.
- [6] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *arXiv:2303.17651*, 2023.
- [7] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” *arXiv:2303.11366*, 2023.
- [8] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, “Power to the People: The Role of Humans in Interactive Machine Learning,” *AI Magazine*, 2014.

- [9] Anthropic, “How We Built Our Multi-Agent Research System,” 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>.
- [10] M. Fowler, “Mocks Aren’t Stubs,” 2007. <https://martinfowler.com/articles/mocksArentStubs.html>.
- [11] Python Software Foundation, “ctypes — A Foreign Function Library for Python.” <https://docs.python.org/3/library/ctypes.html>.
- [12] cocotb developers, “cocotb: A Coroutine-Based Co-Simulation Testbench Environment.” <https://www.cocotb.org/>.
- [13] Applitools, “Visual AI and Visual Regression Testing,” Applitools (industry documentation). <https://applitools.com/>.
- [14] D. Edge et al., “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” *arXiv:2404.16130*, 2024.
- [15] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” *arXiv:2310.08560*, 2023.
- [16] lyogavin, “AirLLM: Layer-by-Layer Inference for Large Models on Small GPUs.” <https://github.com/lyogavin/airllm>.
- [17] A. Zandieh et al., “TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate,” *arXiv:2504.19874*, 2025.
- [18] Qwen Team, “Qwen3,” Alibaba, 2025. <https://qwenlm.github.io/blog/qwen3/>.
- [19] Google DeepMind, “Gemma 3 Model Card,” 2025. https://ai.google.dev/gemma/docs/core/model_card_3.